PROJECT SYNOPSIS

Blue-Green Deployment using Docker & Nginx

using Docker, Nginx, Node.js, Git & GitHub

Student Name	Registration Number
Utkarsh Tiwari	**12315839**

1. Problem Statement

In modern software systems, deploying updates without interrupting users is a major challenge. Traditional deployment methods often require stopping the application, which leads to downtime, poor user experience, and potential business loss.

Additionally, if a newly deployed version contains errors, rolling back to a previous stable version can be time-consuming and risky.

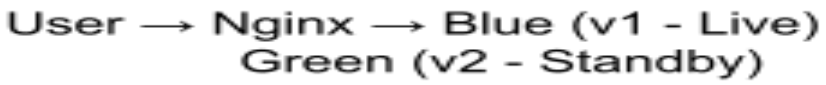
This project addresses these issues by implementing a **Blue-Green Deployment strategy**, where two identical environments run simultaneously. This approach enables seamless switching between application versions, ensuring **zero downtime** and **instant rollback capability**

2. Tools and Technologies Used

Tool / Technology	Purpose
Docker	Containerization of application versions
Nginx	Traffic routing and load balancing
Node.js	Lightweight application for demonstration
Git & GitHub	Version control and project hosting
GitHub	Version control & pipeline triggering

3. System Architecture

The system consists of three main components:

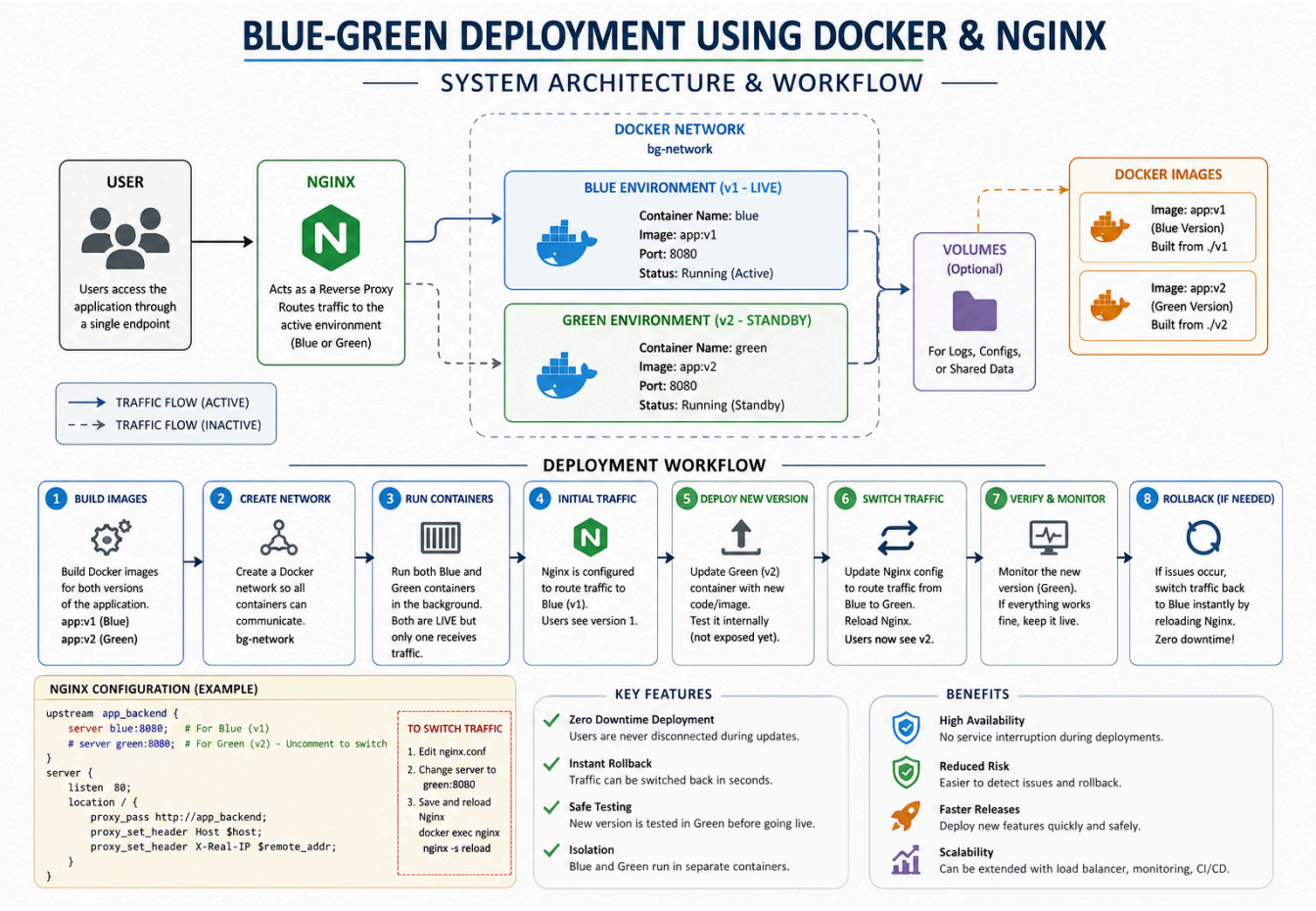


- **Blue Environment:** Current active version serving users
- **Green Environment:** New version deployed in parallel
- **Nginx:** Routes user traffic to either Blue or Green

Traffic is switched dynamically without stopping the application.

4. Workflow Explanation

Step 1	Two versions of the application (v1 and v2) are containerized using Docker.
Step 2	Both containers (Blue and Green) are deployed and connected via a Docker network.
Step 3	Nginx is configured as a reverse proxy to route user requests to the active environment (Blue).
Step 4	Users access the application through Nginx, which forwards requests to the Blue container.
Step 5	When a new version is ready, the Green container is updated and tested in the background.
Step 6	Nginx configuration is updated to switch traffic from Blue to Green without downtime.
Step 7	If the new version works correctly, Green becomes the active environment.
Step 8	If any issue occurs, traffic is instantly switched back to Blue (rollback).



5. Expected Results

- Zero downtime during application deployment
- Seamless switching between application versions
- Instant rollback mechanism in case of failure
- Improved system reliability and availability
- Improved system reliability and availability

6. Conclusion

This project demonstrates the implementation of a **Blue-Green Deployment strategy** using Docker and Nginx, which ensures continuous availability of applications during updates.

By maintaining two parallel environments and controlling traffic through Nginx, the system eliminates downtime and simplifies rollback procedures. This approach reflects real-world DevOps practices used in high-availability systems.

The project highlights key DevOps principles such as **automation readiness, scalability, fault tolerance, and reliability**, making it a strong foundation for advanced deployment pipelines.

Why This Project Stands Out

- Implements real-world deployment strategy (Blue-Green)
- Ensures zero downtime and quick rollback
- Uses industry tools like Docker and Nginx
- Demonstrates practical DevOps knowledge
- Easily extendable to CI/CD and cloud environments

7. Future Scope

- Integration with **CI/CD pipelines (GitHub Actions)** for automated deployment
- Deployment on **cloud platforms (AWS EC2)**
- Addition of **health checks and monitoring systems**
- Scaling using load balancing and container orchestration